

♥ Java ♥

Open-source and
Free!

Random facts

- very popular
- Αρχικά την λέγαν Oak
- 1995 : η πρώτη της έκδοση
- 2009 - σήμερα διατίθεται στο Oracle
- **παρέχει νέα τεχνολογία προγραμματισμού** για εξυπνες συσκευές
- **εμμορφές** στο C, C++, Eiffel, SmallTalk
- **VMS (virtual machines)** για εκτέλεση

Χρήσις σχεδιασμού

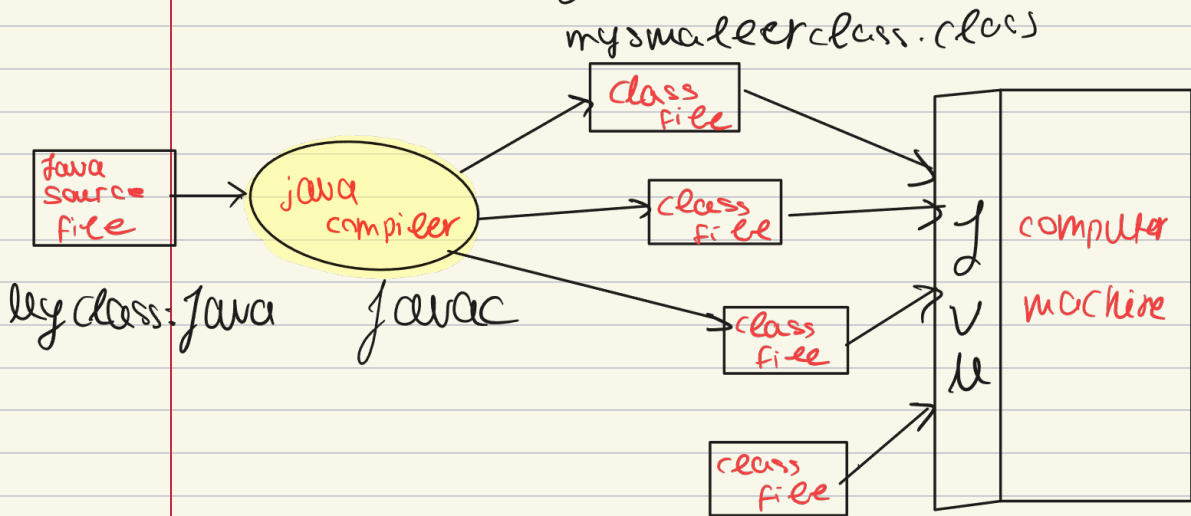
- μεταφερσιμος κωδικας
- διφορούδα λείαση
- εύκολοτερη ανάπτυξη εφαρμογών
- εύκολοτερη ανάπτυξη γραφικών διεπαφών (GUI) ανθρώπου - υπολογιστή
- ασφαλεία

web
↑
telestop
mini-εε
games

Χαρακτηριστικά

- Αντικειμενοστραφής
- Οι κλάσεις της έχουν ιδιότητες βασική κλάση του **object** (by default)
- Οι κλάσεις είναι είτε προσφερόμενες από τον Java, είτε τις γράφει ο

προγραμματισμός είτε είναι υποκώδικας
ή ως Object



κάθε κλάση μεταγλωττίζεται σε
ένα αρχείο κλάσης

→ περιέχει τη πληροφορία για τη
κλάση σε **bytecodes** που είναι εντολές
για τη JVM
Το περιεχόμενο είναι με ακρίβεια
αν 8-bit bytes

η JVM αναγνωρίζει το δυαδικό format
και δεν σχετίζεται απευθείας με το
java

Java

JVM

API

Application Programming
Interface

Συνολικά έσονται 2 ημερίδες που παρέχει η εταιρεία σετ εργαλείων και απεργεί την βασική βιβλιοθήκη της Java σε οργανωμένη σε πακέτα (packages)

IDEs: ECLIPSE, Net Beans

Κωδικός

Ελάχιστος πρόγραμμα σε Java

```
class Name {
```

οι σκιστά από
comment line

```
public static void main (String[] args)
```

```
{
```

```
}
```

μα έξοδο

```
class App {  
    public static void main (String[]  
        args) {  
        System.out.println("Hello World");  
    }  
}
```

υποχρεωτική
μεθόδος

έντονη έξοδος

Στην Java, εάν μια κλάση περιέχει το `public static void main` τότε το όνομα του αρχείου πρέπει να είναι ίδιο με το όνομα της κλάσης αυτής. Στην περίπτωση μας `App.java`

comments /* */ , //

Όπως και στο C και C++ έχουμε μεταβλητές (`string`, `int`, `float`, `char`, `boolean`), οι οποίες πρέπει να έχουν μοναδικά ονόματα

Ο τύπος τους δεν αλλάζει, η τιμή τους μπορεί

`int m = 60;`
↗ identifier

Όταν δεν θέλουμε η τιμή μας μεταβλητής να αλλάξει χρησιμοποιούμε το **final keyword** σαν `const` δηλαδή

```
final int mynum = 15;  
mynum = 20; // Error
```

Primitive types

νέος

types: byte [-128, 127]

boolean - οχι boolean C/C++

- ξεκινάει από μικρό γράμμα
- έχουν τιμή υποχρεωτικά (0)
- είναι ενσωματωμένοι στο Java

non-Primitive types (reference)

Strings, Arrays, Classes, interface

μπορεί να μην είναι αρχική τιμή αν δειχτεί

Η λέξη **var**: αφίνει τον μεταγλωττιστή να καθορίσει τον τύπο της μεταβλητής με βάση την τιμή που της εκχωρείται

var x = 5; αντί για **int x = 5;**

Στοιχες μέθοδοι:

length(), toUpperCase(),
toLowerCase(), indexOf(),
charAt(), equals(), trim()

Πως φτιάχνουμε αντικείμενα;

όχι ~~type t~~ j όπως C++

type **t** = new type C++;

→ μεταβλητή αναφοράς

=> δείκτης στο αντικείμενο του τύπου τους / της κλάσης

=> μπορεί να έχει τιμή null

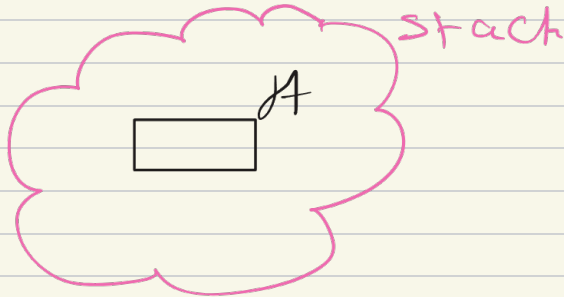
! Αν μια final μεταβλητή δείχνει σε αντικείμενο, η τιμή/κατάσταση του αντικείμενου μπορεί να αλλάξει αλλά η μεταβλητή θα αναφέρεται πάντα στο ίδιο αντικείμενο

• η σε αντικείμενο οποιασδήποτε υποκλάσης

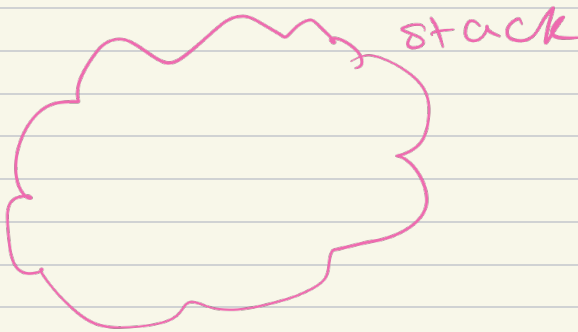
=> μπορεί να πάρει τιμή null μας αλλά η μεταβλητής της κλάσης ή της υποκλάσης της, μέσω αλλαγής

```
class Date {int day, int month,  
int year }
```

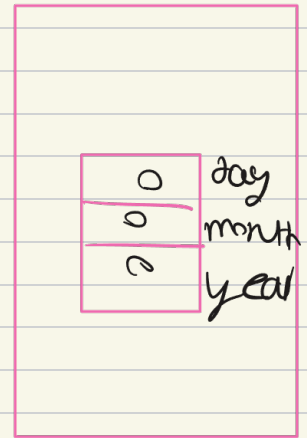
Date H;



new Date();

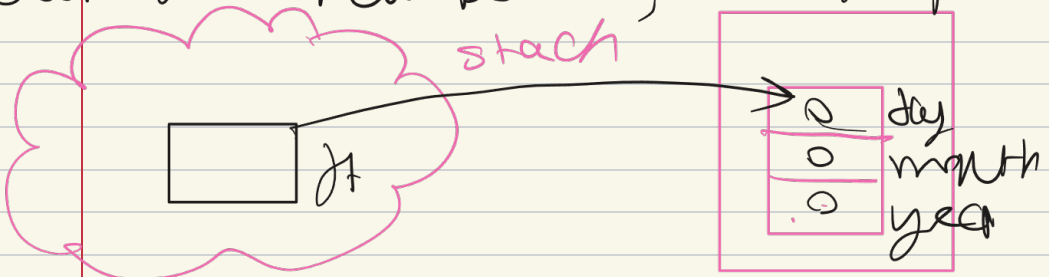


heap



```
Date H = new Date();
```

heap



Λύση στην άσκηση media;

```
private in resources = 1000;  
public c = 'A';
```

this

Αν είχαμε έναν constructor μπορούμε να καλέσουμε έναν άλλον constructor της ίδιας

κλάσης χρησιμοποιώντας το

this (...) και πρέπει να είναι

η πρώτη εντολή

ανακοινώνω
την ύπαρξή της

η οργάνωση των
κλάσεων
περιγράφεται
σε ένα
αρχείο

Πακέτα στο java

χρησιμοποιούνται για να ομαδοποι-
θούν οι κλάσεις. Σημειώστε, το ίδιο
ένα πρόβλημα σε έναν κατανοητό αρ-
χείων

Built-in Packages
(and Java API)

User defined
Packages
(δημιουργούμε
εμείς)

Built-in Packages

```
import package.name.class;  
import package.name.*;
```

ορίζω οι
κλάσεις του

User defined packages

```
└ root  
  └ mypack  
    └ mypackageclass.java
```

ορίσμος πακέτου

```
package mypack;
```

```
class MyClass {
```

```
    public static void main (String[] args) {
```

```
        System.out.println("Αυτός είναι  
το δικό μου πακέτο");
```

```
    }
```

- Δημιουργούν ξεχωριστές ενότητες με ονόματα

- Κάθε κλάση γράφεται σε αρχείο **.java** με το όνομα της και τοποθετείται στο φάκελό με το όνομα του πακέτου

- Τα μεταγλωττισμένα αρχεία **.class** αποθηκεύονται χωριστά ενώ το σύστημα εντοπίζει μέσω της μεθόδου **CLASSPATH**

```
import
```

```
    p1.p2.MyClass
```

εσομένη σε
αλλο πακέτο

↳ ονομα
public κλάσης

~ δεν αντιστοιχεί

Static import : δίνεται να έχουμε
εισαγωγή στατικών μεθόδων ώστε
να χρησιμοποιούνται χωρίς το
όνομα της κλάσης τους

package java.lang περιέχει
σημαντικές κλάσεις (System,
Object) και είναι ορατές σε
κάθε πρόγραμμα χωρίς να
αναφέρεται δηλώνει.

▷ Κάθε πακέτο μπορεί να περι-
λαμβάνει μια ή περισσότερες
μεταγλωττίσεις

Μονάδα μεταγλωττίσεως:

- Διάνωξη πακέτου (package π.π.)
- Αν δίνει η διάνωξη, η κλάση ανήκει
σε ένα unnamed package
- Διάνωξη εισαγωγής
- Διάνωξη τελειώσεως

modifiers C λέδια μέσα οι

public → ορατά και έξω από τις πακέτες που ορίζεται η κλάση

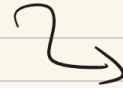
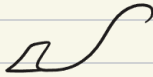
private → ορατά μόνο στην ουσιαστική που ορίζονται

protected → ορατά μέσα από τις πακέτες στις οποίες ορίζεται η κλάση και από τις υποκλάσεις της κλάσης. Μεταξύ όσων ορίζονται (από κλάση και οι υποκλάσεις είναι ορισμένες σε άλλο πακέτο)

by default: ορατά μόνο μέσα σε πακέτο στις οποίες ορίζεται η κλάση

static (ηθδιο) μα μετoβ In ci
με i δια ερθεζεα με τu κτoισι
του οριζεται
→ data hiding

Data abstraction



Abstract class

Δεν μπορεί να δημιουργηθεί αντικείμενο ως πόνι της και αν θέλουμε να τu προσεζoύμε πρέπει να τu κάνουμε μέσω μιας υποκλάσης της

Abstract method

Μπορεί να χρησιμοποιηθεί μόνο ως τu abstract class και δεν έχει σωμα. Το σωμα λoβεζεται αν τu υποκλάση της

```
abstract class Animal {  
    public abstract void sound();  
    public void sleep () {  
        System.out.println("Zzz");  
    }  
}
```

Τηλε κλάση με αφηρημένες μεθoδους
πρέπει υποχρεωτικά να δηλωθεί ως
abstract

Δ μια αφηρημένη μεθοδός δεν μπορεί να συνδυαστεί ως είναι

Δ μια κλάση είναι υποκλάση ενός μα κλάσης

Εμπειρομορφωσις

→ new API

πρῶτα ἱερουργίας → κτῆσιν object

παρὰ ὁδηγῶν ὅτι μα κτῆσιν εἰ-
ναι ὑποκλῆσιν μάς ὁ ὅτις ὑποκλῆ-
σιν εἰς **extends**

```
class Vehicle {
```

```
    protected String brand = "Ford";
```

```
    public void honk() {
```

```
        System.out.println("Tuut, tuut!");
```

```
    }  
}
```

```
class Car extends Vehicle {
```

```
    public static void main(String[] args) {
```

```
        Car c = new Car();  
        c.honk();
```

```
    }  
}
```

ὁδὸς αἱ κτῆσες εἶναι ὑποκλῆσιν εἰς
object

- Αν μια κλάση προσδιορίζεται ως final, τότε δεν μπορούμε να ορίσουμε υποκλάσεις της
- Αν μια μέθοδος προσδιορίζεται ως final, τότε δεν μπορεί να επαναοριστεί από τις υποκλάσεις της
- Αν ένα μέθοδος δηλωθεί final, σημαίνει ότι η τιμή του μεθόδου είναι σταθερή
 - αν το μέθοδος είναι αναμειγνύο σε αντικείμενο, το αντικείμενο μπορεί να αλλάξει τιμή

Η λέξη super

Αναφερόμαστε με αυτή στο γονιμικό κλάση

Ποιον:

- ♥ Να γράψουμε διάφορες μεθόδους της parent class
- ♥ Να γράψουμε τον constructor της parent class

example-access methods

```
class Animal {
```

```
    public void sound() {  
        System.out.println("The animal  
        makes a sound");  
    }  
}
```

```
class Dog extends Animal {
```

```
    public void sound() {
```

```
        super.sound(); // the parent  
        System.out.println("bow wow"); method  
    }
```

```
}
```

```
public class Main {
```

```
    public static void main(String[]  
    args) {
```

```
        Dog myDog = new Dog();
```

```
        myDog.sound();  
    }
```

```
}
```

EXAMPLE OUTPUT: The animal makes a sound

Output: bow wow

example - constructor

```
class Animal {  
    Animal() {
```

```
        System.out.println("Animal is  
        created");  
    }  
}
```

```
class Dog extends Animal {  
    Dog() {
```

```
        super();  
        System.out.println("Dog  
        is created");  
    }
```

```
}
```

```
public class Main {  
    public static void main
```

```
    {  
        Dog myDog = new Dog();  
    }
```

```
}
```

output: Animal is created

Dog is created

Overriding

→ του διαδοχικού

Μη στατικές μεθόδους

Νοια μέθοδος θα επεξεργαστεί
εξαρτάται από τον τύπο του
αντικείμενου (by default συμπερι-
φορά, ενώ στο C++ έπρεπε να
είναι δηλωσούμε ως virtual)

Hiding

→ συνεπαρμένες
με τη κλάση
όχι με τα
αντικείμενα

Στατικές μεθόδους

Σε περίπτωση που η κλάση της
μεθόδου γίνεται μέσω μιας μετα-
βλητής αναφοράς, το, για εμάς
θα καλεστεί συνάρτηση ή υπο-
κλάση) εξαρτάται από τον τύπο
του μεταβλητής και όχι από τον
τύπο του αντικείμενου στον οποίο
αυτή δείχνει

Shadowing

Όταν μια δήλωση αντιστοιχεί
νίξεται σε μια εσωτερική (όχι ο-
ξεία) εμφάνιση (μέσα σε block
ή σε μέθοδο) τότε αυτή **επικαλύπτεται**
τη δήλωση με το ίδιο όνομα, που
υπάρχει στο εξωτερικό εμφάνιση

Interface

Ενας άλλος τρόπος να πετύχαμε abstraction

Είναι μια λίστας μεθόδων που ορίζουν τη συμπεριφορά που πρέπει να έχει μια κλάση που θα υλοποιήσει τον interface. Οι κλάσεις που υλοποιούν τον interface ονομάζονται subclasses.

Παράδειγμα

```
interface Animal {  
    public void sound();  
    public void run();  
}
```

Για να αποκτιθούμε πρόσβαση στις μεθόδους του interface αυτό πρέπει να υλοποιηθεί (implemented) από μια κλάση που θα υλοποιεί τον interface. Η κλάση που υλοποιεί τον interface ονομάζεται subclass.

Το σώμα της μεθόδου παρέχεται από τη κλάση που κάνει τη υλοποίηση.

παράδειγμα

```
interface Animal {  
    public void sound();  
    public void sleep();  
}
```

```
class Pig implements Animal {  
    public void sound() {  
        System.out.println("wi wi");  
    }  
    public void sleep() {  
        System.out.println("zzz");  
    }  
}
```

```
class Main {  
    public static void main(String  
        [] args) {  
        Pig myPig = new Pig();  
        myPig.sound();  
        myPig.sleep();  
    }  
}
```

Παρατηρήσεις

- Δεν μπορούμε να ψελάσουμε αυτήν την `and interface`
- Πρέπει υποχρεωτικά να υλοποιήσουμε όλες τις συναρτήσεις του interface, στα κλάση που καλέει το `implements`
- by default
methods: abstract, public
attributes: public, static, final
- Δεν περιέχει constructors

Στατική μέθοδος

Μια μέθοδος είναι η μέθοδος σχετίζεται με την έννοια της κλάσης γενικότερα και όχι με κάποιο συγκεκριμένο αντικείμενο

Στα στατικά μέθοδοι ανήκουν στην εργασία της κλάσης και υπάρχουν ακόμα και αν δεν έχει δημιουργηθεί κανένα αντικείμενο

τροπή ηθροληθας

- classname.methn?
- μεθω μετoβ?n?us αλφo-
α?

- Αρχικοποιον κλασων
- παραδευμα static
- Διαδικασια αο? (κ?σες,
μετ, τoνo δ'ε?)

